Project Report on – Semantic Spotter

RAG Application Using Langchain

1. Problem Statement:

The goal of the project will be to build a robust generative search system capable of effectively and accurately answering questions from various policy documents.

LangChain is used here to build the generative search application.

2. Why Langchain ?

- LangChain --> is an open-source framework that enables to build applications powered by large language models (LLMs).

- LangChain works --> by providing a layer of abstraction between the developer and the LLM. This abstraction layer makes it easier to use the LLM in a variety of ways providing several features that make it easier to build robust and reliable LLM-based applications that can also be LLM agnostic

- One of the key features of LangChain is --> its support for chaining prompts. This means that multiple prompts can be combined together to create more complex and nuanced requests.

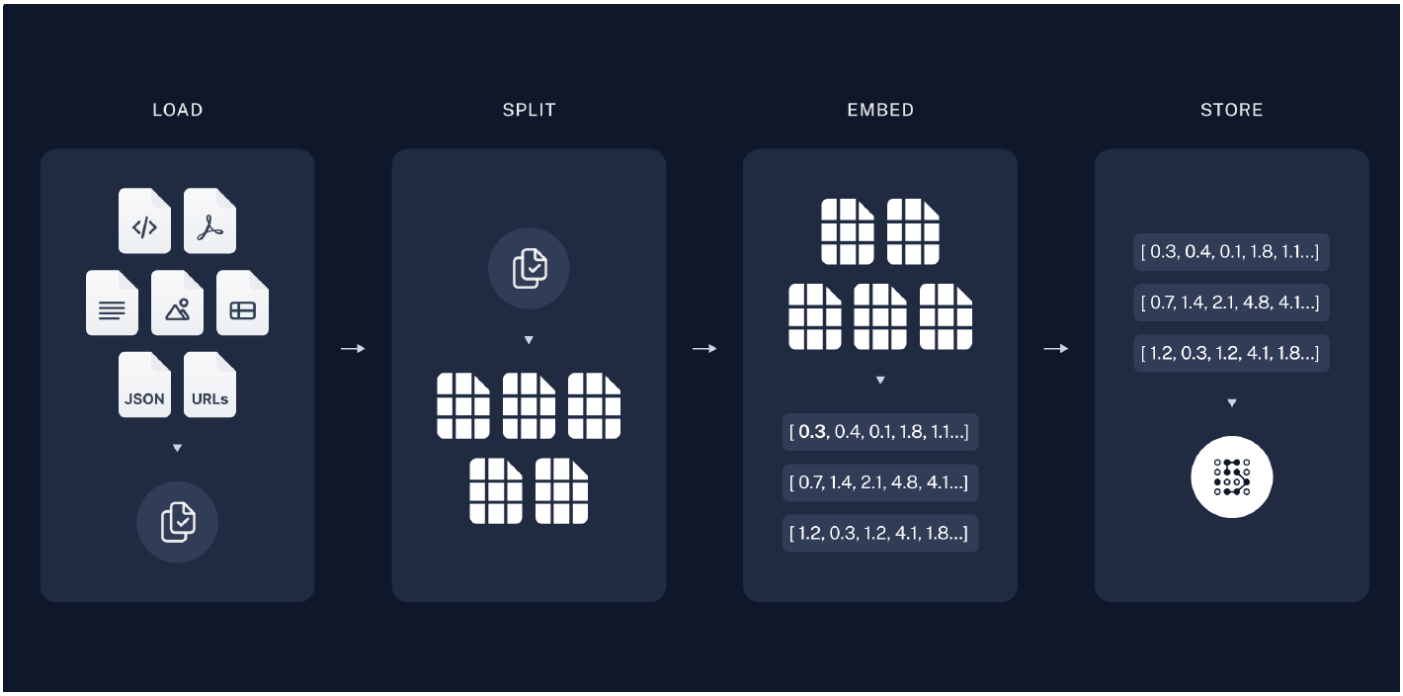
- Another key feature of LangChain is --> its support for modular components. This means that components from different chains can be re-used to create new chains. This can save a lot of time and effort, and it also makes it easier to share and collaborate on chains.

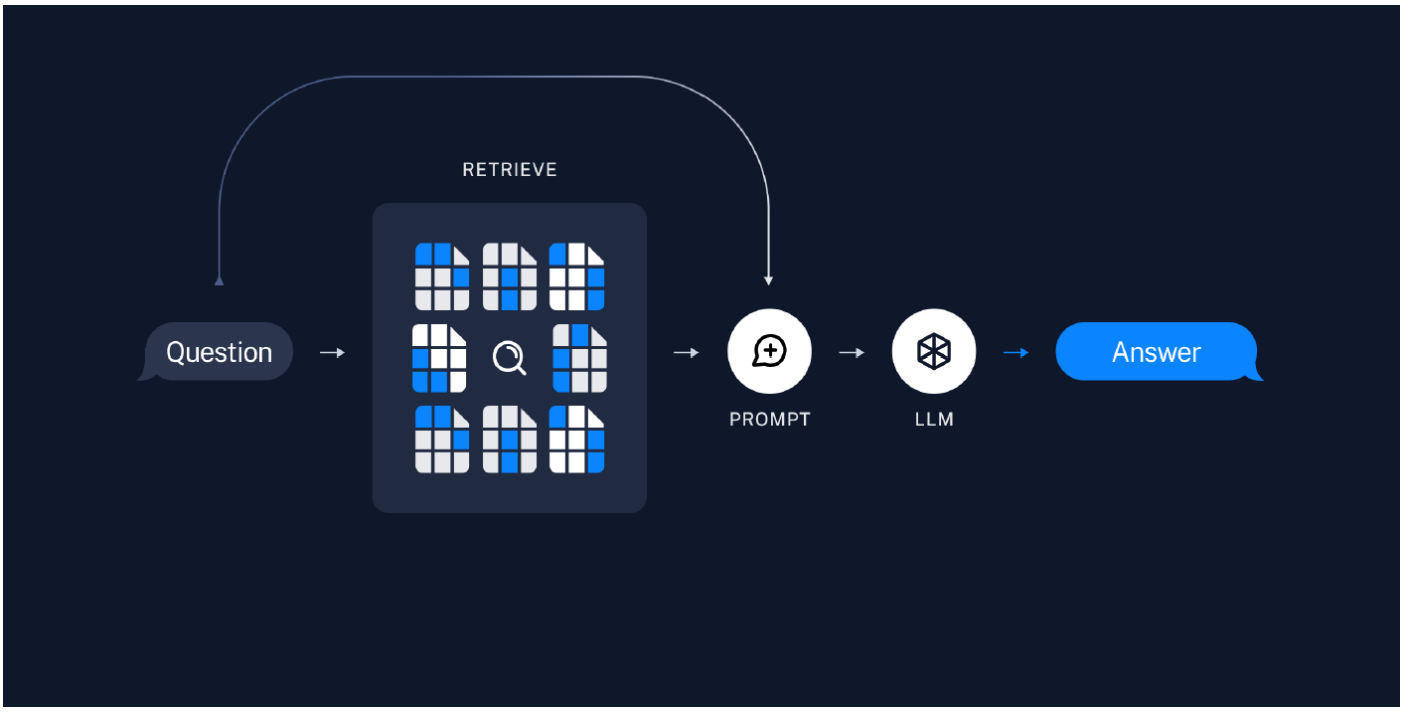
- LangChain offers ---> a suite of tools, components and interfaces that simplify the construction of LLM-centric applications.

- LangChain provides --> an LLM class designed for interfacing with various language model providers, such as OpenAI, Cohere and Hugging Face, that makes it easier to build LLM-agnostic applications by simply switching the language models. This allows to focus on the application logic without delving into the complexities

- Benefits of Using LangChain --> Ease of use, Flexibility, Scalability, Robustness. LangChain provides several features that make it easier to build robust and reliable applications. For example, LangChain supports caching and error handling.

3. System Architecture : Showing Functional flow



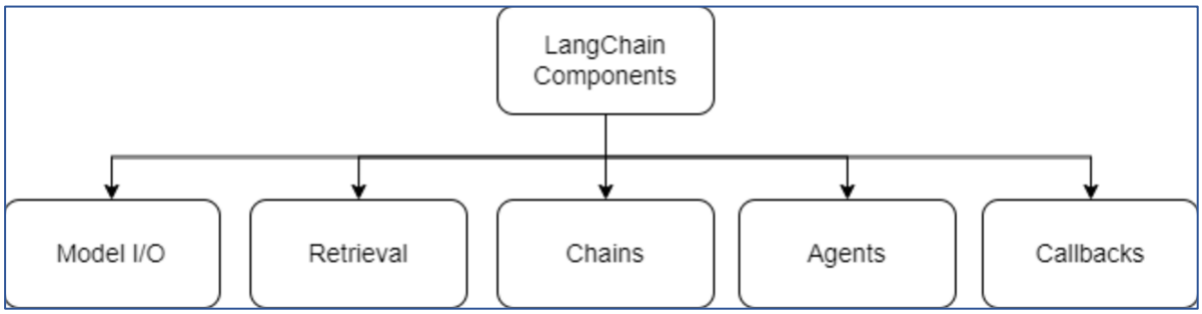


4. Components in Langchain :

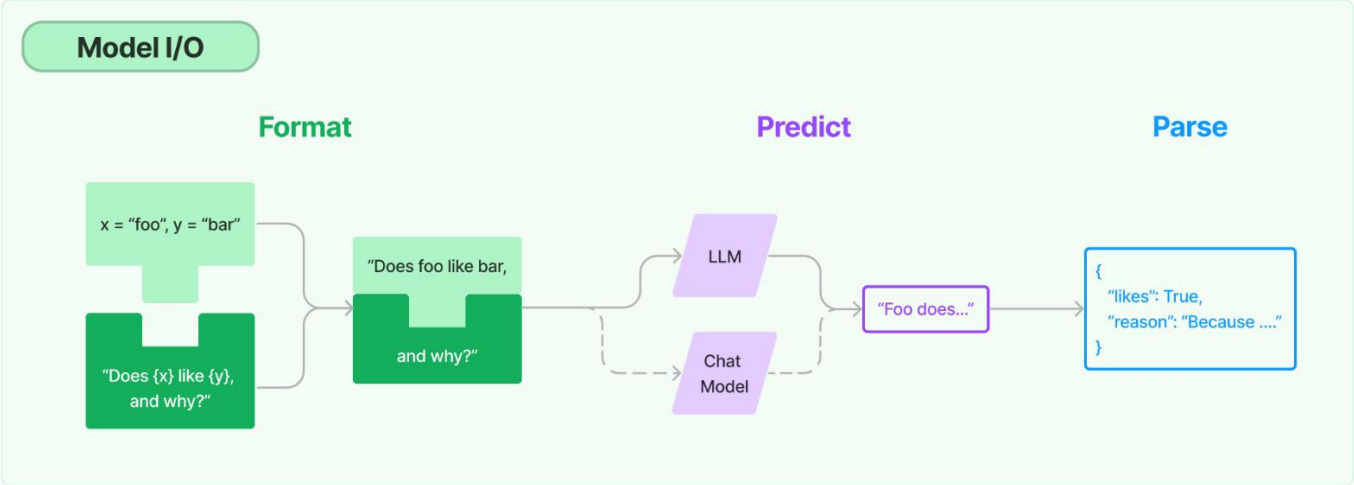
The LangChain framework revolves around the following building blocks:

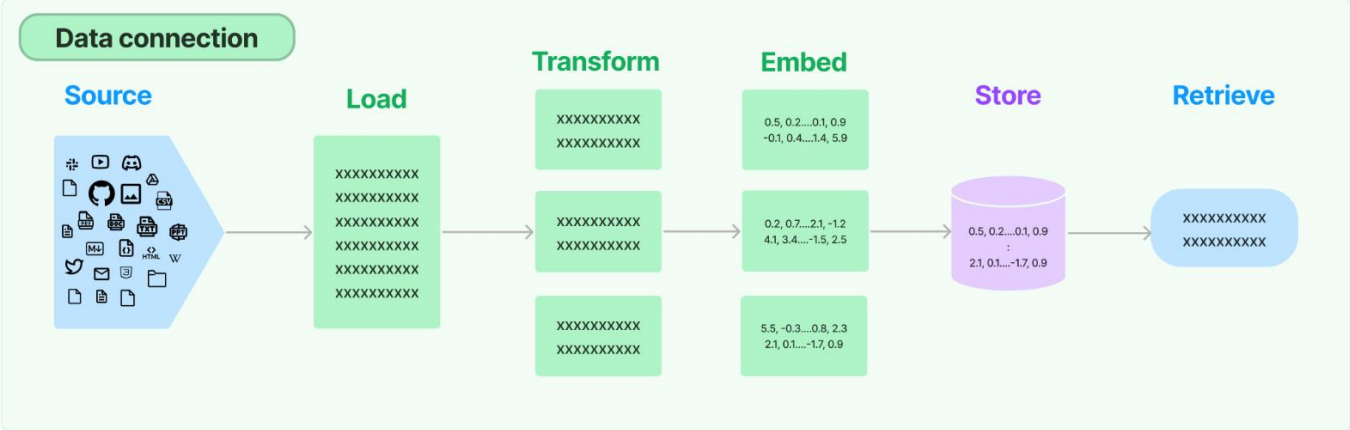
- Model I/O → Interface with language models (LLMs & Chat Models, Prompts, Output Parsers)
- Retrieval → Interface with application-specific data (Document loaders, Document transformers, Text embedding models, Vector stores, Retrievers)
- Chains → Construct sequences/chains of LLM calls
- Memory → Persist application state between runs of a chain
- Agents → Let chains choose which tools to use given high-level directives

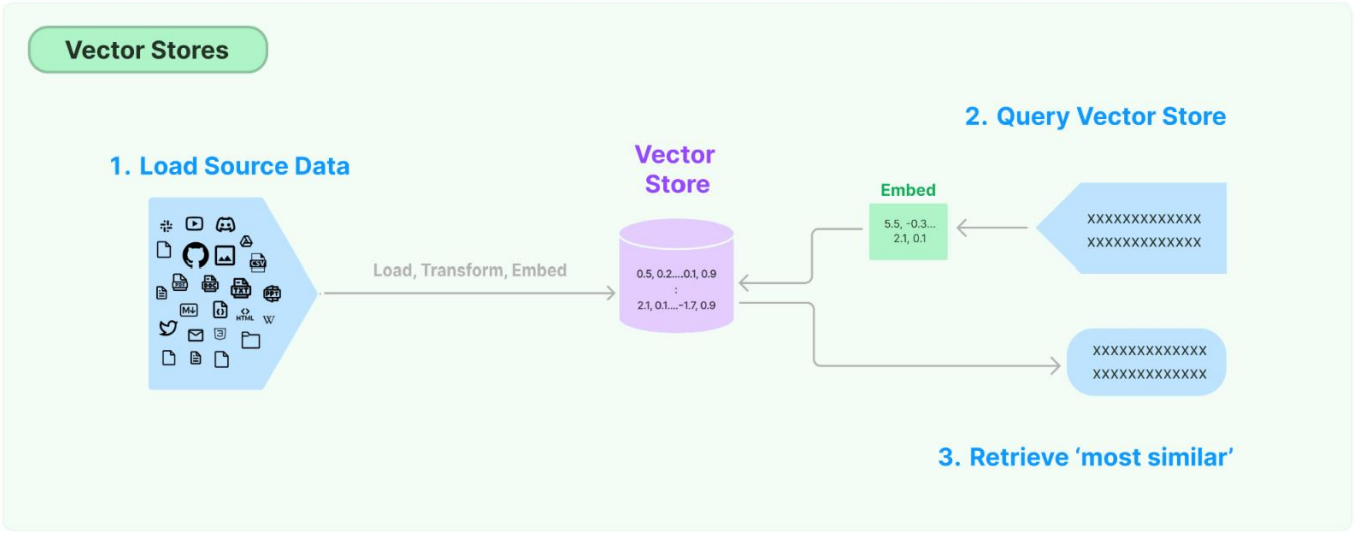
- Callbacks → Log and stream intermediate steps of any chain



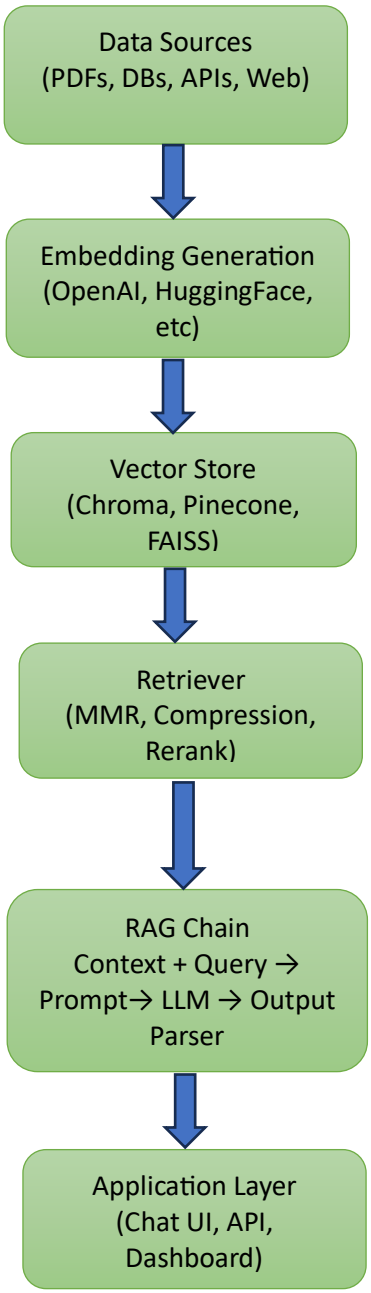
5. Detailed Architecture depicting the Model I/O, Data Connections and Vector Stores :







6. Component based flow and examples of the component types :



7. Packages / Libraries used :

- langchain , openai , chromadb , tiktoken , faiss-cpu , pypdf , docarray
- from langchain_openai import ChatOpenAI, OpenAI
- from langchain_community.document_loaders import PyPDFDirectoryLoader
- from langchain_text_splitters import RecursiveCharacterTextSplitter
- from langchain_openai import OpenAIEmbeddings
- from langchain_community.vectorstores import Chroma
- from transformers import AutoTokenizer, AutoModelForSequenceClassification
- import torch
- from langchain_core.prompts import PromptTemplate
- from langchain_core.runnables import RunnablePassthrough
- from langchain_core.output_parsers import StrOutputParser

8. Implementation Details:

- **Reading & Processing PDF Files** → We will be using LangChain **PyPDFDirectoryLoader** to read and process the PDF files from specified directory.

- **Document Chunking** → We will be using LangChain **RecursiveCharacterTextSplitter**.

    - This text splitter is the recommended one for generic text. It is parameterized by a list of characters. It tries to split on them in order until the chunks are small enough.
    - The default list is ["\n\n", "\n", " ", ""]. This has the effect of trying to keep all paragraphs (and then sentences, and then words) together as long as possible, as those would generically seem to be the strongest semantically related pieces of text.

- **Generating Embeddings** → We will be using **OpenAIEmbeddings** from LangChain package. The Embeddings class is a class designed for interfacing with text embedding models.

    - LangChain provides support for most of the embedding model providers (OpenAI, Cohere) including sentence transformers library from Hugging Face.
    - Embeddings create a vector representation of a piece of text and supports all the operations such as similarity search, text comparison, sentiment analysis etc.
    - The base Embeddings class in LangChain provides two methods: one for embedding documents and one for embedding a query.

- **Store Embeddings In ChromaDB** → In this section we will store embedding in **ChromaDB**. LangChain provides CacheBackedEmbeddings .
    - However, due to the fact that , sometimes the CacheBackedEmbeddings do not get exposed in the current wheel of the LangChain , I have backed the embedding by using a fallback cache mechanism , which implements a “In-Memory cache” using Dictcache function and an embedding wrapper function with caching mechanism “DictCacheEmbeddings”.

- **Retrievers** → Retrievers provide Easy way to combine documents with language models.
  - A retriever is an interface that returns documents given an unstructured query. It is more general than a vector store.
  - A retriever does not need to be able to store documents, only to return (or retrieve) them.
  - Retriever stores data for it to be queried by a language model. It provides an interface that will return documents based on an unstructured query.
  - Vector stores can be used as the backbone of a retriever, but there are other types of retrievers as well.
  - There are many different types of retrievers, the most widely supported is the VectoreStoreRetriever.
  - I have used **db.as_retriever** in this implementation .

- **Re-Ranking with a Cross Encoder** → Re-ranking the results obtained from the semantic search will sometime significantly improve the relevance of the retrieved results.

  - This is often done by passing the query paired with each of the retrieved responses into a cross-encoder to score the relevance of the response with respect to the query.

  - ContextualCompressionRetriever , CrossEncoderReranker, HuggingFaceCrossEncoder are some of the best classes provided by LangChain for re-ranking , however , due to the fact that , sometimes the LangChain.retriever modules do not expose these latest classes and hence I have used a fallback mechanism of "**Manual Cross-Encoder Reranking Pipeline**" .

  - This implementation shows how to manually rerank retrieved documents using HuggingFace transformers when LangChain's built-in reranking classes (e.g., ContextualCompressionRetriever, HuggingFaceCrossEncoder) are not available.

  - It fetches candidate documents via a vector retriever (MMR search with score threshold), then applies a cross-encoder model to score (query, doc) pairs and sort them by relevance.

  - The top-N reranked documents are returned to provide richer, more context-aware inputs for downstream RAG chains.

- **Chains** → LangChain provides Chains that can be used to combine multiple components together to create a single, coherent application.
  - For example, we can create a chain that takes user input, formats it with a PromptTemplate, and then passes the formatted response to an LLM. We can build more complex chains by combining multiple chains together, or by combining chains with other components.
  - In this implementation , I have used **PromptTemplate**, as langchain (or) langchain hub did not expose the rag-prompt class in the current version .
  - And a **Composable RAG Chain :**
    - ✓ This pipeline --> wires together a retriever, prompt, LLM, and output parser using LangChain's Runnable interface.
    - ✓ The retriever ---> fetches relevant documents and formats them into context, while the query passes through unchanged via RunnablePassthrough.

- ✓ The prompt --> injects both context and question into the LLM, and
- ✓ The StrOutputParser --> converts the model's response into a plain string answer.

9. Data Source :

- Documents → Multiple Insurance policy documents have been used as data sources for this implementation which can be found in the repository link .
- Nature of data → Structured legal/insurance contract text, spanning policy terms, eligibility, premiums, termination, and conversion rights.

10. Challenges & Lessons Learned :

1. Cache Implementation → LangChain provides CacheBackedEmbeddings .
   However, due to the fact that , sometimes the CacheBackedEmbeddings do not get exposed in the current wheel of the LangChain , I have backed the embedding by using a fallback cache mechanism , which implements a "In-Memory cache" using Dictcache function and an embedding wrapper function with caching mechanism "DictCacheEmbeddings".

2. Re-ranking → ContextualCompressionRetriever , CrossEncoderReranker, HuggingFaceCrossEncoder are some of the best classes provided by LangChain for re-ranking, however , due to the fact that , sometimes the LangChain.retriever modules do not expose these latest classes and hence I have used a fallback mechanism of "Manual Cross-Encoder Reranking Pipeline" .

3. Chains → rag-prompt class would have been a good option to choose from Langchain. However, neither langchain nor langchain hub exposed rag-prompt class, so I had to implement this using a composable Rag chain mechanism.

11. Execution :

Execution details are given in the Readme file place in the repository below .

https://github.com/ShaktiKP/Semantic_Spotter.git

This repo contains :

1. semantic-spotter-langchain-notebook_Shakti_Priya_Kulkarni.ipynb → Jupyter notebook
2. "Policy Documents" folder containing 7 insurance policy documents
3. README file → containing the steps for execution

Readme content for reference:

- Open "semantic-spotter-langchain-notebook_Shakti_Priya_Kulkarni.ipynb" file in Jupyter Notebook.
- Place the Policy documents and the OpenAI API key file in the same directory and set that directory path in the "folder_path" variable accordingly .
- Run the cells